UCS761 : Lab 7 - From Numbers to Vision: Building, Breaking, Comparing

This assignment integrates everything from **Lab 1 to Lab 7**.

You are expected to:

- Build models **from scratch**
- Count parameters **manually**
- Implement **forward and backward propagation**
- Compare **activations**
- Compare **optimizers**
- Compare **depth**
- Compare **dense vs convolutional structures**
- Plot, interpret, and analyze results

You may use:

- `numpy`
- `matplotlib`

You may NOT use:

- PyTorch / TensorFlow / Keras
- Scikit-learn neural network modules
- Any prebuilt CNN layers
- Any automatic differentiation tools

You must write all learning logic yourself.

GLOBAL RULES (Applies to All Parts)

1. Data Splitting (Mandatory)

You must split your dataset into:

$$70\% \text{ Training}, \quad 15\% \text{ Validation}, \quad 15\% \text{ Test}$$

- Training set → used for learning parameters
- Validation set → used for model selection and comparison
- Test set → used **only once** after final model selection

You are not allowed to tune hyperparameters using the test set.

2. Metrics (Mandatory)

Binary classification threshold:

$$\hat{y}_{bin} = 1[\hat{y} \geq 0.5]$$

Accuracy:

$$\text{Accuracy} = \frac{1}{N}\sum_{i=1}^{N} 1[\hat{y}_i = y_i]$$

Gradient norm (Frobenius norm):

$$\|\nabla W^{(l)}\|_F = \sqrt{\sum_i \sum_j \left(\frac{\partial L}{\partial W_{ij}^{(l)}}\right)^2}$$

3. Required Plots

For every experiment:

- Training Loss vs Epoch
- Validation Loss vs Epoch
- Training Accuracy vs Epoch
- Validation Accuracy vs Epoch

After final model selection:

- Report **Test Accuracy**
- Report **Test Loss**

4. Numerical Stability

For BCE:

$$\hat{y} \leftarrow \text{clip}(\hat{y}, \epsilon, 1-\epsilon)$$

$$\epsilon = 10^{-8}$$

PART 1: Deep Networks on Numeric Data

Dataset (Common for All Students)

Generate:

$$x_1 \sim \text{Uniform}(-2, 2)$$

$$x_2 \sim \text{Uniform}(-2, 2)$$

Target:

$$y = \begin{cases} 1 & \text{if } x_1^2 + x_2^2 > 1.5 \\ 0 & \text{otherwise} \end{cases}$$

Minimum dataset size:

$$N = 3000$$

Split into:

- Train (70%)
- Validation (15%)
- Test (15%)

Task 1A — Build Three Architectures (Dense)

Build:

1. 2-layer network
2. 5-layer network
3. 10-layer network

Constraints:

- ≥ 4 neurons per hidden layer
- 1 output neuron (sigmoid)
- Fully connected

Hidden activations:

- Sigmoid
- ReLU

Output activation:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Task 1B — Parameter Calculation (Mandatory)

Per layer:

$$(n_{in} \times n_{out}) + n_{out}$$

Total:

$$\sum_{\text{layers}} (n_{in} \times n_{out} + n_{out})$$

You must show full derivation in your PDF.

Task 1C — Optimizer Comparison

For each architecture and activation:

Train using:

SGD

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

Momentum

$$v_t = \beta v_{t-1} + \eta \frac{\partial L}{\partial w}$$

$$w \leftarrow w - v_t$$

Use:

$$\beta = 0.9$$

Total models:

$$3 \times 2 \times 2 = 12$$

Required Reporting (Per Model)

Report:

- Final Training Accuracy
- Final Validation Accuracy
- Final Test Accuracy
- Final Training Loss
- Final Validation Loss
- Final Test Loss

Add-On (Mandatory)

For 2-layer vs 10-layer sigmoid networks:

Plot:

$$\|\nabla W^{(l)}\|_F$$

Compare first vs last layer.

Explain gradient behavior.

Required Analysis

Answer:

- Does increasing depth always improve validation performance?
- What happens to test performance as depth increases?
- Do sigmoid networks degrade faster with depth?
- Does optimizer choice affect deep networks more than shallow ones?
- Does validation performance predict test performance reliably?

PART 2 — From Dense to Convolution

Dataset Option (Choose ONE)

Option A — Synthetic 8×8 Images (Recommended)

Class 0: vertical center line

Class 1: horizontal center line

Add Gaussian noise:

$$X \leftarrow X + \mathcal{N}(0, \sigma^2)$$

Use:

$$\sigma = 0.1$$

Minimum size:

$$N = 3000$$

Split into:

- Train 70%
- Validation 15%
- Test 15%

Option B — MNIST (0 vs 1)

If chosen:

- Use only digits 0 and 1
- Same 70/15/15 split

Task 2A — Dense Baseline

Flatten image.

If 8×8:

$$n_{input} = 64$$

Compute dense parameters manually.

Train and report:

- Train/Val/Test metrics

Task 2B — Build CNN From Scratch

Your CNN must include:

- Convolution
- ReLU
- Pooling
- Flatten
- Dense output (sigmoid)

Convolution:

$$\text{out}(i,j) = \sum_{m=0}^{F-1} \sum_{n=0}^{F-1} X(i+m, j+n) K(m,n)$$

You must implement:

- Sliding window manually
- Parameter sharing
- Backpropagation through convolution

Task 2C — Parameter Comparison

Dense:

$$(n_{input} \times n_{hidden}) + n_{hidden}$$

CNN:

$$(F \times F \times C_{in}) \times C_{out} + C_{out}$$

Explain scaling difference.

Task 2D — Structural Experiments

Train:

- CNN with pooling
- CNN without pooling
- CNN with dropout
- CNN without dropout

Dropout:

$$A' = A \cdot M$$

$$M \sim \text{Bernoulli}(p)$$

Report:

- Train/Val/Test metrics
- Overfitting comparison

Required Analysis

Answer:

- Which model generalizes better on test set — dense or CNN?
- Does pooling improve test performance?
- Does dropout reduce validation-test gap?
- How does parameter count relate to test accuracy?
- Does CNN scale better with larger image size?

PART 3 — Optimizer Behavior in CNN

Take best CNN (based on validation performance).

Train using:

- SGD
- Momentum
- Adam

Adam:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

$$w \leftarrow w - \eta \frac{m_t}{\sqrt{v_t} + \epsilon}$$

Use:

$$\beta_1 = 0.9, \quad \beta_2 = 0.999, \quad \epsilon = 10^{-8}$$

Report:

- Train/Val/Test performance
- Convergence speed
- Stability of loss

MASTER RESULT TABLE (Mandatory)

Model	Depth	Activation	Optimizer	Parameters	Train Acc	Val Acc	Test Acc

Notes must explain the behavior observed.

FINAL REFLECTION (Compulsory)(No GPT writing)

Answer clearly:

- Where did training fail due to structure?
- Where did optimizer matter more than activation?
- Where did activation matter more than depth?
- What causes gradient shrinkage?
- Why does CNN generalize better than dense?
- Why does dropout reduce overfitting?
- When does depth hurt test performance?
- Did validation always predict test performance correctly?

Submission Format

Push the folowing in Lab7 on git:

- Python code (.py)
- PDF report
- All plots
- Parameter derivations
- Gradient norm plots
- Master result table

Evaluation Criteria

Marks will be given for:

- Correct forward/backward implementation
- Correct parameter counting
- Correct gradient implementation
- Clear separation of train/val/test
- Proper experimental comparison

- Logical analysis
- Clean readable code

Blind copying without interpretation will receive zero.

Final Statement

This assignment is not about making models run.

It is about understanding:

- Representation
- Structure
- Optimization
- Generalization
- Stability

If signals cannot travel,

learning cannot generalize.

That is the core idea.